Universidad del Valle de Guatemala

Facultad de Ingeniería

Departamento de Ingeniería Mecatrónica, Electrónica y Biomédica

Digital II

Proyecto Viedojuego

Escape From UVG

Juan Antonio Rodríguez Pineda - 221593

Gerardo Lopez -

Guatemala, April 25, 2026

1 Descripción del Proyecto

El siguiente proyecto es una consola de videojuegos que implementa diferentes dispositivos electrónicos vistos durante el curso. Nuestro videojuego llamado Escape From UVG es un juego competitivo que simula un laberinto donde los personajes compiten para ver quien sale primero del laberinto. El ganador se decide en un duelo del mejor de 3 haciendo mostrando el ganador.

En este informe encontrará a detalle la lógica y arquitectura del videojuego, los dispositivos electronicos implementados, los protocolos de comunación y el resultado final del proyecto.

2 Esquemas de conexión

La arquitectura y conexiones realizadas se presentan en la siguiente figura.

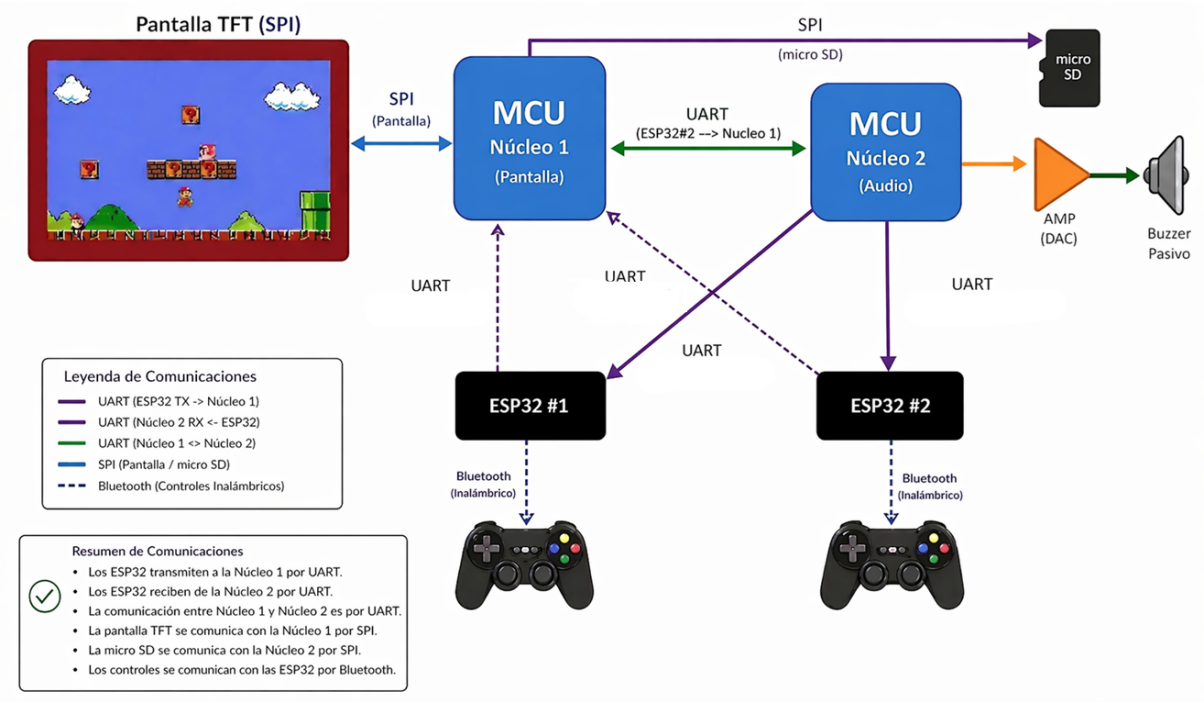

Figure 1: Diagrama de conexión de la consolo

Basicamente la Nucleo 1 era la encargada de manejar todo la información, gráficos y lógica del juego. Está utilizaba una memoria SD como medio de almacenamiento y recibia por medio de interrupciones los comandos de los controles por UART. También

cada que ocurría "eventos" la nucleo 1 le envía comandos a la nucleo 2 para reproducir ciertas canciones o efectos de sonido.

3 Comunicación con controles

Para la interacción del usuario con el videojuego, se utilizó un sistema inalámbrico basado en el protocolo Bluetooth, gestionado por un módulo externo y procesado mediante comunicación serial en la placa principal.

3.1 Transmisor/Receptor: ESP32

El módulo ESP32 actúa como host Bluetooth, utilizando la librería **Bluepad32** para sincronizar mandos de PS4/PS5. Una vez recibidos los estímulos de los botones y joysticks, el ESP32 empaqueta la información en una cadena de texto y la envía hacia la Nucleo-F446RE a través de **UART**. A su vez la ESP32 recibia información por medio de UART de la nucleo2 esto debido a que la nucleo utilizada para la pantalla es la nucleo F401RE que no posee tantos UART. Esta le envía datos a la ESP32 para hacer que los motores del control vibrarán.

3.2 Receptor: Nucleo-F446RE

La placa Nucleo recibe los datos mediante interrupciones para no bloquear la ejecución de los gráficos. Se configuraron dos puertos UART distintos para permitir el juego local de dos personas de manera simultánea.

3.2.1 Captura de datos por Interrupción

Se implementó el *Callback* `HAL_UART_RxCpltCallback` para llenar buffers circulares. Cuando se detecta un carácter de fin de línea (`\n`), se activa una bandera de "nuevos datos" (`newData`).

Listing 1: Recepcion por interrupciones para Jugador 1 y 2

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    // USART6 - Jugador 1
```

```
    if (huart->Instance == USART6) {
        if (rx_char6 == '\n') {
            bufferU6[idx6] = '\0';
            idx6 = 0;
            newDataU6 = 1;
        } else {
            if (idx6 < sizeof(bufferU6) - 1) bufferU6[idx6++] =
               rx_char6;
        }
        HAL_UART_Receive_IT(&huart6, &rx_char6, 1);
    }
    // USART1 - Jugador 2
    if (huart->Instance == USART1) {
        if (rx_char == '\n') {
            bufferU1[idx] = '\0';
            idx = 0;
            newDataU1 = 1;
        } else {
            if (idx < sizeof(bufferU1) - 1) bufferU1[idx++] =
               rx_char;
        }
        HAL_UART_Receive_IT(&huart1, &rx_char, 1);
    }
}
```

3.2.2 Procesamiento y Lógica de Control

Una vez recibida la cadena (ej. `"P1:U1R0L0D0..."`), se utiliza `sscanf` para parsear los valores booleanos de cada botón. La lógica de control se divide en dos estados principales: el **Menú** (donde se navega con el cursor) y el **Nivel Activo** (donde se mueven los personajes).

Listing 2: Procesamiento de entrada y reset de juego

```
if (newDataU6) {
    newDataU6 = 0;
```

```
    int U, R, L, D, X, A, B, Y;
    sscanf(bufferU6, "P1:U%dR%dL%dD%dX%dA%dB%dY%d", &U, &R, &L, &
       D, &X, &A, &B, &Y);

    // LOGICA DE RESET: Boton B regresa al menu principal
    if (B == 1 && nivelA != 0) {
        Score1 = 0; Score2 = 0; // Reset de variables globales
        PJ1.px = 128; PJ1.py = 208; // Posicion inicial
        nivelA = 0;
        Draw_Level_Background(0);
        transmitSonic(0, 0, 'N', '0'); // Notificar al modulo de
           audio
    }

    // NAVEGACION EN MENU
    if (nivelA == 0) {
        if (D == 1) { cursor.py += 16; }
        if (U == 1) { cursor.py -= 16; }
        if (X == 1) { F_Start = 1; } // Confirmar seleccion
    }
    // MOVIMIENTO EN JUEGO
    else {
        if (D == 1) { PJ1.py += 16; }
        if (U == 1) { PJ1.py -= 16; }
        if (L == 1) { PJ1.px -= 16; }
        if (R == 1) { PJ1.px += 16; }
        moveFlag = 1;
    }
}
```

Este sistema garantiza que ambos jugadores tengan una respuesta inmediata (baja latencia) y permite funciones globales como el "Reset" total del estado de los enemigos (*spikes* y *shells*) y marcadores al presionar un solo botón desde cualquier punto del juego.

4 Sonido

La implementación del sistema de audio representó uno de los mayores desafíos técnicos del proyecto, principalmente debido a la necesidad de sincronizar la música ambiental con los efectos de sonido sin interrumpir la lógica principal del videojuego.

4.1 Arquitectura del Sistema de Audio

Para la generación de sonido se utilizaron ambos canales del **DAC** (Digital to Analog Converter). Se determinó que la solución más estable era separar las tareas por tipo de salida:

- **Canal 1 (Música de fondo):** Utiliza una arquitectura no bloqueante basada en interrupciones de hardware.
- **Canal 2 (Efectos de sonido):** Utiliza una arquitectura bloqueante para efectos rápidos y variaciones de frecuencia (*sweeps*).

4.2 Música de Fondo No Bloqueante

Para evitar que el flujo del juego se detuviera durante las canciones, se implementó la función `Song_Start`. Esta función no utiliza `HAL_Delay`, sino que se apoya en el **Timer 2**, el cual genera una interrupción cada 1 ms para actualizar la nota actual y su duración.

Listing 3: Implementación de motor de música no bloqueante

```
void Song_Start(uint32_t *notes, uint16_t *duration, uint16_t
   size0)
{
    song_notes    = notes;
    song_duration = duration;
    song_size     = size0;
    song_index    = 0;
    note_tick     = 0;
    note_phase    = 0;
    song_playing  = 1;

    // Cálculo inicial de tiempos para la primera nota
```

```
    uint16_t dur = duration[0];
    note_ticks_on  = dur * 90 / 100;
    note_ticks_off = dur * 10 / 100;
    if (note_ticks_off < 25) note_ticks_off = 25;

    playToneDAC(notes[0], 1); // Canal 1 activo
}
```

4.3 Efectos de Sonido (FX)

Para los efectos de sonido, como el movimiento del personaje (`footstep`) o colisiones (`blockHit`), se utilizaron funciones bloqueantes. Aunque estas detienen brevemente el CPU, su corta duración (milisegundos) no afecta la percepción de fluidez. Esta decisión se tomó para simplificar la gestión del **DMA** y evitar conflictos al reconfigurar los periféricos en tiempo real.

Listing 4: Efecto de sonido de pasos con síntesis por frecuencia

```
void footstep()
{
    // 1. IMPACTO: Ataque rápido ascendente
    for(uint32_t arr = 800; arr < 1400; arr += 60) {
        playToneDAC(arr, 2);
        HAL_Delay(1);
    }

    // 2. PESO: Caída de frecuencia (decay)
    for(uint32_t arr = 1400; arr > 300; arr -= 40) {
        playToneDAC(arr, 2);
        HAL_Delay(2);
    }

    // 3. TEXTURA: Generación de ruido blanco para fricción
    generateNoise();
    HAL_DAC_Start_DMA(&hdac, DAC_CHANNEL_1, (uint32_t*)Ynoise,
```

```
        size, DAC_ALIGN_12B_R);
    HAL_Delay(20);

    playToneDAC(0, 2); // Silencio tras el paso
}
```

4.4 Sincronización y Resultados

La coexistencia de ambos sistemas permitió que el videojuego mantuviera una atmósfera sonora constante mediante temas musicales complejos, mientras reaccionaba de forma inmediata a las acciones del jugador. El uso de interrupciones para la música garantizó que el procesamiento de la pantalla y los controles no sufrieran degradación de rendimiento.

5 Gráficos del videojuego

5.1 Pantalla ili9341

La pantalla tiene una resolución de 320x240 pixeles, ambos son multiplos de 16. Se tomo la decision desde bastante temprano que el juego estaria dividido en una cuadricula de 20x15, cada bloque de 16x16 pixeles. Esto hizo claro que cada personaje ocuparia solo un bloque, permitiendo amplio espacio para moverse en pantalla.

La funcion utilizada para dibujar en la pantalla fue:

```
LCD_Bitmap(x,y,w,h,sprite);
```

Esta funcion requiere de 5 parametros, coordenadas en x y y, anchura, altura y el sprite que se dibujara. Por estos motivos, se creó un struct como el que esta a continuacion.

```
struct P_objeto {
        uint16_t px;
        uint8_t py;
        uint8_t w;
        uint8_t h;
};
```

Esto permitia crear objetos con atributos diseñados para la pantalla. Tambien, son atributos utiles para las colisiones. Para no tener que tomar demasiado poder de procesamiento con el dibujo, se dibujaron niveles de 320x240 para dibujar solo una vez al inicio de ellos.

5.2 SD

La gestión de memoria externa a través de la tarjeta SD es fundamental para el renderizado de escenarios complejos sin saturar la memoria Flash del microcontrolador. Para ello, se implementó la función `Draw_Level_Background`, la cual se encarga de extraer mapas de bits en formato binario (*.bin*) y dibujarlos píxel por píxel en la pantalla ILI9341.

5.2.1 Gestión del Bus SPI y Multiplexación

Dado que tanto la pantalla como el lector SD comparten el bus **SPI1**, la función implementa una lógica de conmutación dinámica de velocidades (Prescalers). La tarjeta SD opera de forma estable con un *BaudRatePrescaler* de 8, mientras que la pantalla permite una comunicación más agresiva con un *Prescaler* de 2 para maximizar la tasa de refresco.

El algoritmo sigue estos pasos principales:

1. **Selección de Archivo:** Dependiendo del parámetro `nivel`, se define el nombre del archivo (ej. *niv1.bin*, *creditos.bin*).

2. **Montaje y Apertura:** Se utiliza la biblioteca FatFs para montar el sistema de archivos y abrir el recurso en modo lectura.

3. **Lectura por Filas:** Para optimizar el uso de la memoria RAM, no se carga la imagen completa. Se utiliza un *buffer* de 320 píxeles (`rowBuf`) que representa una fila horizontal de la pantalla.

4. **Renderizado:** Por cada fila leída, se desactiva el *Chip Select* (CS) de la SD, se reconfigura el SPI a alta velocidad, y se envían los datos a la pantalla mediante el protocolo de comando de ventana (`SetWindows`).

5.2.2 Implementación del Código

Listing 5: Función para dibujar fondo desde SD

```
void Draw_Level_Background(uint8_t nivel) {
    char dbg[80];
    char filename[20];

    // Selección de archivo según el nivel
    if (nivel == 4) sprintf(filename, "creditos.bin");
    else if (nivel == 5) sprintf(filename, "HS.bin");
    else sprintf(filename, "niv%d.bin", nivel);

    // 1. Inicialización de SPI Lento para SD
    hspi1.Init.BaudRatePrescaler = SPI_BAUDRATEPRESCALER_8;
    HAL_SPI_Init(&hspi1);

    // 2. Montar sistema de archivos FATFS
    FATFS fs_lvl;
    FRESULT res = f_mount(&fs_lvl, "/", 0);

    if (res != FR_OK) return; // Error de montaje

    // 3. Abrir archivo del nivel
    FIL imgFile;
    res = f_open(&imgFile, filename, FA_READ | FA_OPEN_EXISTING);
    if (res != FR_OK) {
        f_mount(NULL, "", 0);
        return;
    }

    static uint16_t rowBuf[320]; // Buffer de una fila

    for (int row = 0; row < 240; row++) {
        // --- LECTURA DE SD (SPI Lento) ---
        HAL_GPIO_WritePin(SD_SS_GPIO_Port, SD_SS_Pin,
           GPIO_PIN_RESET);
```

```
        UINT bytesRead = 0;
        f_read(&imgFile, rowBuf, 640, &bytesRead);
        HAL_GPIO_WritePin(SD_SS_GPIO_Port, SD_SS_Pin,
           GPIO_PIN_SET);

        // --- DIBUJO EN LCD (SPI Rápido) ---
        hspi1.Init.BaudRatePrescaler = SPI_BAUDRATEPRESCALER_2;
        HAL_SPI_Init(&hspi1);

        SetWindows(0, row, 319, row);
        HAL_GPIO_WritePin(LCD_CS_GPIO_Port, LCD_CS_Pin,
           GPIO_PIN_RESET);
        HAL_GPIO_WritePin(LCD_DC_GPIO_Port, LCD_DC_Pin,
           GPIO_PIN_SET);

        for (int i = 0; i < 320; i++) {
            uint8_t lo = rowBuf[i] & 0xFF;
            uint8_t hi = rowBuf[i] >> 8;
            HAL_SPI_Transmit(&hspi1, &lo, 1, 1);
            HAL_SPI_Transmit(&hspi1, &hi, 1, 1);
        }
        HAL_GPIO_WritePin(LCD_CS_GPIO_Port, LCD_CS_Pin,
           GPIO_PIN_SET);

        // Volver a velocidad SD para la siguiente iteración
        hspi1.Init.BaudRatePrescaler = SPI_BAUDRATEPRESCALER_8;
        HAL_SPI_Init(&hspi1);
    }

    f_close(&imgFile);
    f_mount(NULL, "/", 0);
}
```

6 Lógica del videojuego

Cuando el juego esta activo, hay cuatro posibles acciones que el usuario puede tomar y es moverse en 4 direcciones. Por esta razon, una de los aspectos de logica del juego mas importantes eran las colisiones.

El nucleo recibia un comando, sea arriba, abajo, izquierda o derecha y se sumaria o restaria 16 a la posicion x o y del jugador correspondiente. En esta nueva posicion, se hace un chequeo utilizando una funcion especial llamada ColCheck.

```
uint8_t ColCheck(struct P_objeto o1, struct P_objeto o2)
{
        if ((o1.px + o1.w - 1>= o2.px) && (o2.px + o2.w - 1>= o1.
           px) && (o1.py + o1.h - 1>= o2.py) && (o2.py + o2.h -
           1>= o1.py))
        {
                return 0;
        }
        else{
                return 1;
        }
}
```

Esta funcion chequea si dos bojetos colisionaron. Para ahorrar procesamiento, solo se chequea colisiones cuando un personaje se mueve y solo chequea colisiones contra objetos en la misma coordenada en Y. De esta manera, no se hacen chequeos con objetos alejados del jugador. Para ejemplificar, el chequeo de colisiones para el nivel 1:

```
            if (nivS == 1)
            {
                switch (j2.py)
                {
                case 224:
                    coli &= ColCheckj2(j2, par3);
                    break;
```

```
case 208:
case 192:
    coli &= ColCheckj2(j2, blq1_j1);
    break;

case 176:
    coli &= ColCheckj2(j2, blq2_j1);
    break;

case 160:
    coli &= ColCheckj2(j2, blq3_j1);
    coli &= ColCheckj2(j2, blq4_j1);
    break;

case 144:
    coli &= ColCheckj2(j2, blq4_j1);
    coli &= ColCheckj2(j2, blq6_j1);
    break;

case 128:
    coli &= ColCheckj2(j2, blq4_j1);
    coli &= ColCheckj2(j2, blq5_j1);
    break;

case 112:
    coli &= ColCheckj2(j2, blq7_j1);
    coli &= ColCheckj2(j2, blq11_j1);
    break;

case 96:
    coli &= ColCheckj2(j2, blq9_j1);
    coli &= ColCheckj2(j2, blq11_j1);
    break;
```

```
        case 80:
            coli &= ColCheckj2(j2, blq9_j1);
            coli &= ColCheckj2(j2, blq10_j1);
            coli &= ColCheckj2(j2, blq11_j1);
            coli &= ColCheckj2(j2, blq12_j1);
            break;

        case 64:
            coli &= ColCheckj2(j2, blq11_j1);
            coli &= ColCheckj2(j2, blq13_j1);
            break;

        case 48:
            coli &= ColCheckj2(j2, blq13_j1);
            coli &= ColCheckj2(j2, blq15_j1);
            break;

        case 32:
            coli &= ColCheckj2(j2, blq14_j1);
            coli &= ColCheckj2(j2, blq17_j1);
            break;

        case 16:
            coli &= ColCheckj2(j2, blq19_j1);
            coliW &= ColCheckj2(j2, star1);
            break;

        case 0:
            coli &= ColCheckj2(j2, par1);
            break;
        }
    }

    if (coli)
```

```
            {
                M2_F = 1;
            }
            else
            {
                j2.px = j2o.px;
                j2.py = j2o.py;
            }
```

De esta manera, si se detecta una colision al moverse, se cancela el movimiento y se regresa a las coordenadas anteriores.

Tambien hay condiciones especiales que avanzan el nivel, en este caso, coliW. ColiW activa una bandera especial que avanza el nivel y este es el ciclo principal del juego.

7 Resultados

Como resultado se obtuvo un juego de 3 niveles con movimiento, sonido y estimulos en el control implementados. Con errores lógicos dentro del juego y con algunos problemas de funciones bloqueantes dentro del Audio.

8 Resultados y Repositorio

8.1 Link a repositorio

- **Video de funcionamiento:** github.com/Juan-Rodriguez3/Digital_2/ProyectoVideoJuegoF401R

8.2 Demostración en Video

- **Video de funcionamiento:** Demostración del Proyecto en YouTube